Per-Second Error Detection Model on the Kinect CSV Data

A new model built on the preprocessed CSV (model_input_kinect.csv): trained on Groups 1A + 2A, tested on Group 3. Code, outputs and what they mean.

13 October 2026

Status: exploratory analysis (not part of the pre-registered confirmatory analysis)

What this document is

Question	Answer
Data used	The **preprocessed CSV file** `model_input_kinect.csv` from the `input_data` folder. One row per Kinect frame, with the expert labels already attached.
Task	Build a model on this CSV: one model per exercise, trained and validated on Groups 1A + 2A (split by person), tested on Group 3. Kinect first; BlazePose later with the same steps.
Code	All code in this document (steps 1–14) is new, written in a separate notebook.
The `keraal_pipeline` folder	Shared **code only, no data**, so it was **not run** here. It was used only as a reference: its joint-angle formulas were reused, and its published angle values were used to check this code (step 10).
Status	Exploratory. These are first results, not final conclusions (see Limitations).

How this differs from `keraal_pipeline`

Same research topic and same participants, but a different analysis:

	`keraal_pipeline`	**This model**
Data	Original KERAAL files (`raw/` folder)	Preprocessed CSV `model_input_kinect.csv` (`input_data` folder)
Trained on → tested on	**Group 3 → 1A + 2A** (acted errors → real patients)	**1A + 2A → Group 3** (real data → acted errors)
What is predicted	One label per **whole recording**	One label per **second**
Exercises	Main (pre-registered) test on CTK only	CTK, ELK and RTK, one model each
Result	Pre-registered hypotheses not supported (no clear signal)	Clear signal on Group 3 (AUC 0.71–0.77, p = 0.001)

What overlaps

- **The angle idea.** The joint-angle formulas and the heading rotation are reused from the pipeline's `features.py` and `preprocess.py`, on purpose. Step 10 checks this code's angles against the pipeline's numbers. That is a check, not a repeat.
- **One finding.** Like the pipeline, this work finds that raw camera positions mostly capture the person and the camera, not the error. It is found here in a different setting (per second, opposite direction), which supports the pipeline's finding.

What is new

- The first model built on the preprocessed CSV data.
- Error detection second by second, which is what a real-time feedback app would need.

- Results for all three exercises.
- The Error1 vs Error3 difference in CTK, which a merged “E13” class cannot show.

The two analyses complement each other. The pipeline asks whether acted errors teach a model to recognise real errors. This model asks whether real errors teach a model to recognise acted ones, second by second.

Before you read: the exercises and the scores

The three exercises

CTK, ELK and RTK are the names of the three rehabilitation exercises in the KERAAL dataset. Each exercise gets its **own model**, because each has different movements and different errors.

Code	Exercise	What the person does	Error the model looks for
CTK	Hiding face	Raises both arms up in front of the face	Arms not raised high enough (Error1 + Error3)
ELK	Flank stretch	Bends the trunk to the side, with one arm over the head	Body not bent enough to the side (Error2)
RTK	Torso rotation	Turns the upper body to the side	Arms not raised enough (Error1) or rotation not big enough (Error2)

The score: ROC-AUC

Almost every number in this report is a **ROC-AUC** score. It answers one question:

*If I pick one **error** example and one **correct** example at random, how often does the model give the error the higher “error” score?*

ROC-AUC	Meaning
0.5	A coin flip. The model cannot tell errors and correct movements apart.
about 0.6	Weak: slightly better than chance.
about 0.7–0.8	Clear: the model ranks errors higher most of the time.
1.0	Perfect: every error scores higher than every correct example.
below 0.5	Worse than a coin flip. In this report that only happens for single people, mostly by chance (small numbers), or in step 7 because of how results were pooled.

Example. CTK 0.732 means: the model ranked an error above a correct example 73% of the time.

How to read Table 1 (testing inside Groups 1A and 2A)

- Each value is a **within-person ROC-AUC**. One person is hidden, the model is trained on everyone else, and then it is tested on the hidden person's own seconds. This is repeated for every person and the results are averaged.
- Each **row** is a different input given to the model: raw positions, normalized positions, angles, and the final version (angles + 5-second window). Comparing the rows shows **which input works**.
- **Reading it:** positions stay near 0.5 (no signal). Angles raise CTK to about 0.72. The final version gives CTK 0.73, ELK 0.58 and RTK 0.60: good for CTK, only slightly above chance for ELK and RTK.

How to read Table 2 (final test on Group 3)

- **ROC-AUC:** measured per **recording**, on three people the model never saw. Example: ELK 0.768 means an error recording got a higher error score than a correct recording 77% of the time.
- **Chance level (95th percentile):** the highest score that luck alone usually reaches. The labels were shuffled at random 1,000 times, and 95% of those random tries scored below this value (e.g. 0.592 for ELK). A real score above it is unlikely to be luck.
- **p-value:** the share of random shuffles that did as well as the real result. 0.001 means about 1 in 1,000, so the result is very unlikely to be luck. Below 0.05 is the usual cut-off.
- **Works for each of the 3 people?** The ROC-AUC computed separately for P1, P2 and P3. RTK works for all three (0.81–0.93), CTK for all three (0.67–0.84), ELK for two of them (P2 = 0.48, a coin flip).

In one sentence: the model finds errors clearly above chance in all three exercises on Group 3, and is most reliable for RTK and CTK.

Summary

Question. Can a model trained on expert-labelled real rehabilitation data (Groups 1A and 2A) recognise movement errors in new people, using only Kinect skeleton data? We built one model per exercise (CTK, ELK, RTK) that predicts, for every second of a recording, whether the movement is *correct* or an *error*. We checked it inside 1A/2A by holding out one person at a time, and then tested it once on Group 3 (three new people who act out errors).

Features (what the model sees)	CTK	ELK	RTK
Raw joint positions (camera coordinates)	0.567	0.547	0.673
Normalized joint positions (centred, scaled)	0.548	0.495	0.505
Joint angles, no heading alignment	0.660	0.540	0.596
Joint angles, heading aligned	0.723	0.526	0.471
Final: aligned angles + absolute angles + 5-second context	**0.732**	**0.582**	**0.597**

Table 1. Validation inside 1A/2A: mean within-person ROC-AUC (leave-one-participant-out). 0.5 = coin flip, 1.0 = perfect.

Group 3 test (recording level)	CTK	ELK	RTK
ROC-AUC	**0.713**	**0.768**	**0.768**
Chance level (95th percentile of 1,000 label shuffles)	0.578	0.592	0.561
p-value	0.001	0.001	0.001
Works for each of the 3 people?	yes (0.67–0.84)	2 of 3 (P2 = 0.48)	yes (0.81–0.93)

Table 2. Final test on Group 3, run once with the frozen model.

Main messages

- Raw or normalized joint coordinates carry almost no error information for a new person. Joint angles that match each error's clinical description (arm elevation, trunk side tilt, torso rotation) do.
- Errors are about the *peak* of a repetition, so the model needs to see a few seconds around each second (5-second context window).
- On Group 3 the model ranks error recordings above correct ones 71–77% of the time, clearly above chance (p = 0.001).

- In CTK the model detects Error1 but hardly Error3, even though both are described as “arms not raised enough”. This needs a closer look before the two are merged.

1 · Background and design decisions

Data

We use the preprocessed file `model_input_kinect.csv` from the `input_data` folder (237,733 rows, 196 columns). One row is one Kinect frame. The data uses 15 frames per second, so 15 rows make one second. There are 21 label/meta columns and 175 skeleton columns (25 joints × x, y, z position and qx, qy, qz, qw orientation).

Group	Who	Labels	Role here
1A	6 patients (R1–R6)	Experts marked the exact seconds of each error	Training + validation
2A	6 healthy adults (S1–S6)	Same as 1A	Training + validation
3	3 healthy people acting out errors (P1–P3)	One label for the whole recording, from the file name	Test (used once)

Design decisions

- Start with Kinect. BlazePose (webcam) is the final goal but harder, so it comes later with the same steps.
- One model per exercise: CTK (hiding face), ELK (flank stretch), RTK (torso rotation).
- Train and validate on 1A + 2A, always split by person, so no one is in both training and validation.
- Test on Group 3. Its people never appear in 1A/2A, so the test set does not overlap with training.

Problems we handled in the code

Problem	Why it matters	What we did
Group 3 labels cover whole recordings	1A/2A labels are per second; G3 is not	Score Group 3 per recording (average of its seconds)
Some error types never appear in 1A/2A for an exercise	A model cannot predict a class it never saw	Per exercise, keep only error types that exist in training
Camera position differs in Group 3	A model on raw coordinates can learn the camera, not the movement	Use body angles, after rotating each recording to face the camera
Errors are rare (5–16% of seconds)	“Always correct” already looks 85–95% accurate	class_weight='balanced'; report recall/F1 and ROC-AUC, never accuracy

Note on direction

The pre-registered analysis in `keraal_pipeline` trains on Group 3 and tests on 1A/2A. This work goes the other way (train on real data, test on acted data), so all results here are exploratory.

2 · Key terms

Term	Meaning in this project
Feature	A number the model uses as input (e.g. the highest arm angle in a second).
Target	What the model must predict: 0 = correct, 1 = error.
Label leakage	When the answer sneaks into the input (e.g. a label column used as a feature). The score looks great but means nothing.
Leave-one-participant-out	Hold out one person, train on everyone else, predict the held-out person; repeat for everyone.
Recall (error)	Of the real error seconds, the share the model caught.
Precision (error)	Of the seconds the model called “error”, the share that really were errors.
F1	One number that balances recall and precision (0 = bad, 1 = perfect).
Macro-F1	Average of the F1 for “correct” and the F1 for “error”, so both count equally.
ROC-AUC	If you pick one random error and one random correct example, how often does the model give the error the higher error probability? 0.5 = coin flip, 1.0 = perfect. Does not depend on any cut-off.
Permutation test	Shuffle the labels many times and recompute the score, to see how high it can go by luck. The p-value is the share of shuffles that did at least as well as the real result.

The steps below were run in order in one Jupyter notebook in VS Code. Each step uses variables from the steps before it, so the data is loaded only once.

Setup. In VS Code: open the project folder, create a notebook (`.ipynb`), choose the Python kernel, and run `pip install pandas scikit-learn ipykernel` in the terminal.

Step 1 · Load the data and check the columns

Why. Before building anything, confirm the file matches the metatable and that no label column is mixed in with the skeleton features (label leakage). Features are chosen *by name*, not by position.

CODE

```
import pandas as pd

# Load the data and make "group" all text (fixes the 3 vs "3" problem)
# Use the full path: right-click the CSV in VS Code -> Copy Path
data = pd.read_csv("/YOUR/FULL/PATH/input_data/model_input_kinect.csv", low_memory=False)
data["group"] = data["group"].astype(str)

# The 21 label/meta columns from the metatable PDF (never used as model input)
meta_columns = [
    "sample_id", "group", "exercise_code", "participant_key", "trial_id",
    "frame_idx", "time_sec", "evaluation", "error_type", "body_part",
    "severity", "label_source", "label_quality", "in_annotated_span",
    "skeleton_truncated", "second_idx", "sec_error_type", "sec_body_part",
    "sec_error_frac", "sec_n_frames", "sec_boundary",
]

# Check 1: are all meta columns in the file?
missing = [c for c in meta_columns if c not in data.columns]
print("Missing meta columns:", missing)

# Features = every column that is not a meta column
feature_columns = [c for c in data.columns if c not in meta_columns]
print("Number of features:", len(feature_columns))

# Check 2: every feature name should end like a Kinect joint value
endings = ("_x", "_y", "_z", "_qx", "_qy", "_qz", "_qw")
strange = [c for c in feature_columns if not c.endswith(endings)]
print("Unexpected feature names:", strange)
print("First 7 features:", feature_columns[:7])

# Check 3: number of people in each group
print(data.groupby("group")["participant_key"].nunique())
```

CODE EXPLAINED

Code	What it does
`import pandas as pd`	Loads pandas, the Python library for tables. *as pd* lets us write the short name *pd*.
`pd.read_csv(path, low_memory=False)`	Reads the CSV file into a table (called a *DataFrame*). *low_memory=False* makes pandas read the whole file before deciding each column's type, so one column cannot end up with a mix of numbers and text.
`data["group"].astype(str)`	Turns every value in the group column into text. Without it, Group 3 was stored both as the number 3 and the text "3" and showed up as two different groups.
`meta_columns = [ ... ]`	A Python *list*: an ordered collection of items in square brackets. Here it holds the names of the 21 label/info columns from the metatable.
`[c for c in meta_columns if c not in data.columns]`	A *list comprehension*: a short loop that builds a new list. Read it as: "for every name c in meta_columns, keep c if it is not a column of the file". An empty result [] means nothing is missing.
`[c for c in data.columns if c not in meta_columns]`	Same idea the other way round: every column that is not a label column is a skeleton feature.
`len(feature_columns)`	Counts how many items a list has.
`c.endswith(endings)`	Gives True if the text c ends with one of the endings, e.g. "ElbowLeft_x" ends with "_x" → True. A label column like "severity" would give False and be caught.
`feature_columns[:7]`	*Slicing*: the first 7 items of the list (positions 0 to 6).
`data.groupby("group")["participant_key"].nunique()`	*groupby* splits the rows into groups (1A, 2A, 3). For each group, *nunique()* counts how many *different* participant names appear.

OUTPUT

```
Missing meta columns: []
Number of features: 175
Unexpected feature names: []
First 7 features: ['SpineBase_x', 'SpineBase_y', 'SpineBase_z', 'SpineBase_qx', 'SpineBase_qy',
                   'SpineBase_qz', 'SpineBase_qw']
group
1A    6
2A    6
3     3
```

What it means. All three checks pass. The file has every meta column from the PDF, exactly 175 skeleton columns, and no oddly named column, so no label leaked into the input. The groups have the expected 6, 6 and 3 people.

Troubleshooting. A `FileNotFoundError` means Python is looking in a different folder. Using the full path (VS Code: right-click the file → Copy Path) always works.

Step 2 · Filter the data and count what is left

Why. Training (1A/2A) should use only clean, expert-checked seconds. For the test set (Group 3) we count recordings, because its labels cover whole recordings. The counts decide which classes each exercise's model can learn.

CODE

```
# ---------- Training data: Groups 1A and 2A ----------
train_data = data[data["group"].isin(["1A", "2A"])].copy()                # 1A + 2A only
train_data = train_data[train_data["in_annotated_span"] == "yes"]         # part the expert annotated
train_data = train_data[train_data["sec_n_frames"] == 15]                 # complete seconds only
train_data = train_data[~train_data["evaluation"].isin(["Incomplete", "Motionless"])]
train_data = train_data[train_data["sec_error_type"] != "Other"]          # rare "Other" class
print("Training frames left:", len(train_data))

# One row per second (all 15 frames in a second share the same label)
train_seconds = train_data.drop_duplicates(subset=["sample_id", "second_idx"])
print("Training seconds left:", len(train_seconds))
print(pd.crosstab(train_seconds["exercise_code"], train_seconds["sec_error_type"]))
print(pd.crosstab(train_seconds["participant_key"], train_seconds["sec_error_type"]))

# ---------- Test data: Group 3 ----------
test_data = data[data["group"] == "3"].copy()
test_recordings = test_data.drop_duplicates(subset="sample_id")           # one row per recording
print("Test recordings:", len(test_recordings))
print(pd.crosstab(test_recordings["exercise_code"], test_recordings["error_type"]))
```

CODE EXPLAINED

Code	What it does
`data["group"].isin(["1A", "2A"])`	Checks every row: True if its group is 1A or 2A, otherwise False.
`data[ True/False list ]`	Putting a True/False check inside data[ ] keeps only the rows where it is True. This is how all filtering works in pandas.
`.copy()`	Makes a separate table, so later changes do not affect the original *data*.
`train_data["in_annotated_span"] == "yes"`	== asks "is it equal?" for every row. Keeps frames inside the part the expert really annotated.
`~train_data["evaluation"].isin([...])`	The ~ sign means NOT: keep rows whose evaluation is *not* Incomplete or Motionless.
`!= "Other"`	!= means "is not equal to".
`drop_duplicates(subset=["sample_id", "second_idx"])`	All 15 frames of one second share the same label. This keeps only the first row of each (recording, second) pair, so we can count *seconds* instead of frames.
`pd.crosstab(a, b)`	Counts how often each combination of two columns appears, like a pivot table. Rows = values of a, columns = values of b.

OUTPUT

```
Training frames left: 66422
Training seconds left: 4531

Training seconds per exercise and error type:
sec_error_type  Error1  Error2  Error3  none
CTK                109       6      77  1491
ELK                  0     185       0   931
RTK                 35      46       0  1651

Training seconds per participant and error type:
                Error1  Error2  Error3  none
1A_R1               30      88       0   377
1A_R2               13       0      40   272
1A_R3               15      36      24   424
1A_R4               29      41       0   625
1A_R5                5      48       0   661
1A_R6                0       6      13   853
2A_S1                5      18       0   400
2A_S2                4       0       0   121
2A_S3               14       0       0   141
2A_S4                0       0       0   119
2A_S5                0       0       0    49
2A_S6               29       0       0    31

Test recordings per exercise and error type:
              Error1  Error2  Error3  none
CTK               45      45      45    45
ELK               45      45      45    45
RTK               45      45      45    45
```

What it means.

- About 17% of the 4,531 training seconds are errors, so errors are rare.
- Each exercise has different errors. CTK has Error1 and Error3 (Error2 only 6 s). ELK has only Error2. RTK has small amounts of Error1 (35 s) and Error2 (46 s).
- Errors come from only a few people. S4 and S5 have none, and Error3 appears only for R2, R3 and R6. So the model may partly learn *who* makes errors.
- Group 3 is perfectly balanced: 45 recordings for every exercise × error type.
- 66,422 frames ÷ 15 = 4,428, but 4,531 seconds were counted. A few seconds at the edge of the annotated span lost some frames. Step 3 fixes this.

Step 3 · Make one yes/no target per exercise

Why. With this little error data, a yes/no model (correct vs error) is far more reliable than telling error types apart. For each exercise we keep only error types that exist in training. CTK Error1 and Error3 are merged (both “arms not raised enough”, as in `keraal_pipeline`). RTK Error1 and Error2 are merged because each alone is too small.

CODE

```
# Which error types count as "error" for each exercise
error_classes = {
    "CTK": ["Error1", "Error3"],   # arms not raised enough (merged)
    "ELK": ["Error2"],
    "RTK": ["Error1", "Error2"],   # any error (each alone is too small)
}

# Keep only seconds that still have all 15 frames after filtering
frames_per_second = train_data.groupby(["sample_id", "second_idx"])["frame_idx"].transform("count")
train_data = train_data[frames_per_second == 15]

# Per exercise, keep "none" + that exercise's error types (training and test)
train_parts, test_parts = [], []
for exercise, errors in error_classes.items():
    train_parts.append(train_data[(train_data["exercise_code"] == exercise) &
                                  (train_data["sec_error_type"].isin(["none"] + errors))])
    test_parts.append(test_data[(test_data["exercise_code"] == exercise) &
                                (test_data["error_type"].isin(["none"] + errors))])
train_data = pd.concat(train_parts)
test_data = pd.concat(test_parts)

# Target: 0 = correct, 1 = error
train_data["target"] = (train_data["sec_error_type"] != "none").astype(int)
test_data["target"] = (test_data["error_type"] != "none").astype(int)

train_seconds = train_data.drop_duplicates(subset=["sample_id", "second_idx"])
print(pd.crosstab(train_seconds["exercise_code"], train_seconds["target"]))
test_recordings = test_data.drop_duplicates(subset="sample_id")
print(pd.crosstab(test_recordings["exercise_code"], test_recordings["target"]))
```

CODE EXPLAINED

Code	What it does
`error_classes = { "CTK": [...], ... }`	A *dictionary*: pairs of key → value. Here each exercise (key) points to the list of error types that count as "error" for it.
`groupby(["sample_id", "second_idx"])["frame_idx"].transform("count")`	For every row, counts how many rows share its (recording, second). *transform* gives the answer back for every row, so it lines up with the table. Example: a second that lost 3 frames gives 12.
`for exercise, errors in error_classes.items():`	A loop over the dictionary. Each round gives one key (exercise) and its value (errors).
`(condition A) & (condition B)`	& means AND: both must be True. Each condition needs its own brackets.
`["none"] + errors`	Joins two lists, e.g. ["none"] + ["Error1", "Error3"] → ["none", "Error1", "Error3"].
`pd.concat(train_parts)`	Stacks the three per-exercise tables on top of each other into one table.
`(train_data["sec_error_type"] != "none").astype(int)`	First gives True/False (is it an error?), then *astype(int)* turns True → 1 and False → 0. This 0/1 column is the *target* the model must predict.

OUTPUT

```
Training seconds (0 = correct, 1 = error):
target            0    1
CTK            1430  166
ELK             876  169
RTK            1585   76

Test recordings (0 = correct, 1 = error):
target          0   1
CTK            45  90
ELK            45  45
RTK            45  90
```

What it means. Errors make up 10% of CTK seconds, 16% of ELK and only 5% of RTK, so RTK is the hardest. The Group 3 test sets have 135, 90 and 135 recordings. In Group 3, half to two thirds

of recordings are errors, very different from training.

Step 4 · Turn each second into one row (raw positions)

Why. The model needs one row and one label per second. For each second we summarize the 15 frames of every joint position with mean, std, min and max. Only the 75 position columns are used: Kinect orientations are noisy, and `keraal_pipeline` also uses positions only. 75 × 4 = 300 features.

CODE

```
# Position columns only: 25 joints x (x, y, z) = 75
position_columns = [c for c in feature_columns if c.endswith(("_x", "_y", "_z"))]

def make_second_features(df):
    grouped = df.groupby(["sample_id", "second_idx"])                    # frames -> seconds
    stats = grouped[position_columns].agg(["mean", "std", "min", "max"])  # summary per second
    stats.columns = [f"{col}_{stat}" for col, stat in stats.columns]      # e.g. SpineBase_x_mean
    info = grouped[["participant_key", "exercise_code", "target"]].first()
    return info.join(stats).reset_index()

test_data = test_data[test_data["sec_n_frames"] == 15]                   # complete seconds only
train_sec = make_second_features(train_data)
test_sec = make_second_features(test_data)
print("Training seconds table:", train_sec.shape)
print("Test seconds table:", test_sec.shape)
print("Missing values (train):", train_sec.isna().sum().sum())
print("Missing values (test):", test_sec.isna().sum().sum())
```

CODE EXPLAINED

Code	What it does
`c.endswith(("_x", "_y", "_z"))`	Keeps only position columns (25 joints × 3 = 75) and drops the orientation columns (qx, qy, qz, qw).
`def make_second_features(df):`	*def* creates a reusable *function*. *df* is the table we give it. Everything indented below belongs to the function; *return* sends the result back.
`df.groupby(["sample_id", "second_idx"])`	Groups the frames so that each group is one second of one recording (15 frames).
`.agg(["mean", "std", "min", "max"])`	For every group and every column, computes 4 numbers: the average (mean), how much it varies (std, standard deviation), the lowest (min) and the highest (max). Example: the hand's height over one second.
`f"{col}_{stat}"`	An *f-string*: builds text from variables. With col = "SpineBase_x" and stat = "mean" it gives "SpineBase_x_mean".
`.first()`	Takes the first value in each group. Fine here because participant, exercise and target are the same for all 15 frames.
`info.join(stats).reset_index()`	*join* puts the two tables side by side. *reset_index* turns sample_id and second_idx back into normal columns.
`.shape / .isna().sum().sum()`	*shape* gives (rows, columns). *isna()* marks missing values; summing twice counts them over the whole table.

OUTPUT

```
Position columns: 75
Training seconds table: (4302, 305)
Test seconds table: (5117, 305)
Missing values (train): 0
Missing values (test): 0
             sample_id  second_idx participant_key exercise_code  target  SpineBase_x_mean ...
0  G1A-CTK-R1-Brest-022           0           1A_R1           CTK       0         -0.171231
1  G1A-CTK-R1-Brest-022           1           1A_R1           CTK       0         -0.164218
```

What it means. 4,302 training seconds (exactly the sum from step 3) and 5,117 test seconds from 360 recordings, each with 300 features and nothing missing. The pelvis barely moves within a second (std about 0.01, about 1 cm).

Step 5 · Validate with leave-one-participant-out (raw positions)

Why. Before touching Group 3, check whether the model finds errors in people it has not seen. For each exercise and each person: train a Random Forest on everyone else and predict the held-out person. `class_weight='balanced'` makes rare error seconds count more.

CODE

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import recall_score, precision_score, f1_score

info_columns = ["sample_id", "second_idx", "participant_key", "exercise_code", "target"]
feature_names = [c for c in train_sec.columns if c not in info_columns]

def validate(table, exercise):
    ex = table[table["exercise_code"] == exercise]
    y_true_all, y_pred_all = [], []
    for person in ex["participant_key"].unique():                # each person held out once
        train = ex[ex["participant_key"] != person]
        val = ex[ex["participant_key"] == person]
        model = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                       min_samples_leaf=2, random_state=0, n_jobs=-1)
        model.fit(train[feature_names], train["target"])
        y_pred_all += list(model.predict(val[feature_names]))
        y_true_all += list(val["target"])
    error_rate = sum(y_true_all) / len(y_true_all)
    print(f"{exercise}:  error recall {recall_score(y_true_all, y_pred_all):.3f}"
          f" | error precision {precision_score(y_true_all, y_pred_all, zero_division=0):.3f}"
          f" | error F1 {f1_score(y_true_all, y_pred_all):.3f}"
          f" | macro-F1 {f1_score(y_true_all, y_pred_all, average='macro'):.3f}"
          f" | chance F1 {2 * error_rate / (1 + error_rate):.3f}")

for exercise in ["CTK", "ELK", "RTK"]:
    validate(train_sec, exercise)
```

CODE EXPLAINED

Code	What it does
`RandomForestClassifier(...)`	A *Random Forest* is many decision trees. Each tree learns simple yes/no rules from a random part of the data (e.g. "is the arm angle below 100°?"). For a new example, the trees vote, and the share of "error" votes is the error probability.
`n_estimators=300`	Use 300 trees. More trees give more stable votes.
`class_weight="balanced"`	Errors are rare, so mistakes on error seconds are counted as more costly. Otherwise the model could say "correct" every time and still be right about 90% of the time.
`min_samples_leaf=2`	Each final rule must cover at least 2 examples. Stops trees from memorising single seconds.
`random_state=0, n_jobs=-1`	random_state fixes the randomness so results repeat exactly. n_jobs=-1 uses all processor cores (faster).
`for person in ex["participant_key"].unique():`	Leave-one-participant-out: every person takes one turn as the held-out person. *unique()* lists each person once.
`model.fit(X, y)`	Training: the model learns from the features X and the answers y (0/1).
`model.predict(X)`	Prediction: gives 0 or 1 for each second (error if more than half the trees vote error).
`y_pred_all += list(...)`	Adds this person's predictions to the running list, so all people are scored together at the end.
`recall_score / precision_score / f1_score`	The scores from the key-terms table. *average='macro'* averages the F1 of both classes. *zero_division=0* avoids an error when the model never says "error".
`2 * error_rate / (1 + error_rate)`	Chance F1: if the model called *every* second an error, recall = 1 and precision = error rate p, so F1 = 2p / (1 + p). A useful model must beat this.
`{value:.3f}`	Inside an f-string, :.3f shows a number with 3 decimals.

OUTPUT

```
CTK:  error recall 0.024 | error precision 0.034 | error F1 0.028 | macro-F1 0.467 | chance F1 0.188
ELK:  error recall 0.018 | error precision 0.023 | error F1 0.020 | macro-F1 0.429 | chance F1 0.278
RTK:  error recall 0.000 | error precision 0.000 | error F1 0.000 | macro-F1 0.485 | chance F1 0.088
```

What it means. The model almost never says “error” (recall 0–2.4%), and when it does it is usually wrong. Error F1 is *below* the “call everything an error” baseline for every exercise. The macro-F1 near 0.45 comes almost entirely from the easy majority class. **Raw camera coordinates mostly describe where a person stands and how big they are, not how they move**, so they do not carry over to a new person.

Step 6 · Normalize the skeleton and validate again

Why. Make every body comparable: subtract the pelvis (SpineBase) from every joint (centre) and divide by the person's torso length (scale).

CODE

```
import numpy as np

joints = [c[:-2] for c in position_columns if c.endswith("_x")]       # 25 joint names

def normalize(df):
    df = df.copy()
    pelvis = df[["SpineBase_x", "SpineBase_y", "SpineBase_z"]].values
    shoulder = df[["SpineShoulder_x", "SpineShoulder_y", "SpineShoulder_z"]].values
    df["torso"] = np.sqrt(((shoulder - pelvis) ** 2).sum(axis=1))     # torso length per frame
    torso_per_recording = df.groupby("sample_id")["torso"].transform("median")
    for joint in joints:
        for i, axis in enumerate(["x", "y", "z"]):
            col = f"{joint}_{axis}"
            df[col] = (df[col] - pelvis[:, i]) / torso_per_recording   # centre + scale
    return df

train_sec_norm = make_second_features(normalize(train_data))
test_sec_norm = make_second_features(normalize(test_data))
for exercise in ["CTK", "ELK", "RTK"]:
    validate(train_sec_norm, exercise)
```

CODE EXPLAINED

Code	What it does
`import numpy as np`	NumPy: the library for fast maths on arrays (grids of numbers).
`c[:-2]`	Removes the last 2 characters: "ElbowLeft_x" → "ElbowLeft".
`df[[...]].values`	Turns the selected columns into a NumPy array with shape (rows, 3) = x, y, z per frame.
`np.sqrt(((shoulder - pelvis) ** 2).sum(axis=1))`	3D distance (Pythagoras): difference in x, y, z → square → add → square root. This is the torso length in each frame.
`transform("median")`	The median torso length of each recording, given back on every row. Median ignores occasional Kinect glitches better than the mean.
`for i, axis in enumerate(["x", "y", "z"]):`	*enumerate* gives both a counter i (0, 1, 2) and the item ("x", "y", "z").
`(df[col] - pelvis[:, i]) / torso_per_recording`	pelvis[:, i] is the pelvis x (i=0), y (i=1) or z (i=2). Subtracting it moves the body to the origin; dividing by torso length removes body size.

OUTPUT

```
Normalized training table: (4302, 305)
CTK:  error recall 0.042 | error precision 0.200 | error F1 0.070 | macro-F1 0.504 | chance F1 0.188
ELK:  error recall 0.000 | error precision 0.000 | error F1 0.000 | macro-F1 0.445 | chance F1 0.278
RTK:  error recall 0.000 | error precision 0.000 | error F1 0.000 | macro-F1 0.475 | chance F1 0.088
```

What it means. Only a small gain for CTK (precision 3% → 20%), and none for ELK and RTK. Two explanations are possible: (1) the model sees some signal but its 50% cut-off is too strict, or (2) one second of positions simply does not hold the information. Step 7 tells them apart.

Step 7 · Check for any signal with ROC-AUC (pooled)

Why. ROC-AUC does not depend on the 50% cut-off. Near 0.5 means no signal. Clearly above 0.5 means signal exists.

CODE

```
from sklearn.metrics import roc_auc_score

def validate_auc(table, exercise):
    ex = table[table["exercise_code"] == exercise]
    y_true_all, y_prob_all = [], []
    for person in ex["participant_key"].unique():
        train = ex[ex["participant_key"] != person]
        val = ex[ex["participant_key"] == person]
        model = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                       min_samples_leaf=2, random_state=0, n_jobs=-1)
        model.fit(train[feature_names], train["target"])
        y_prob_all += list(model.predict_proba(val[feature_names])[:, 1])   # P(error)
        y_true_all += list(val["target"])
    print(f"{exercise}:  ROC-AUC {roc_auc_score(y_true_all, y_prob_all):.3f}")
```

CODE EXPLAINED

Code	What it does
`model.predict_proba(X)[:, 1]`	Instead of 0/1, gives probabilities for each class: [P(correct), P(error)]. [:, 1] keeps the second column, P(error).
`roc_auc_score(y_true, y_prob)`	Ranking score. Example: with 1 error second (prob 0.7) and 2 correct seconds (0.4 and 0.8), the error beats 0.4 but not 0.8 → AUC = 1/2 = 0.5. With perfect ranking every error beats every correct second → AUC = 1.

OUTPUT

```
Raw positions:
CTK:  ROC-AUC 0.526
ELK:  ROC-AUC 0.435
RTK:  ROC-AUC 0.494

Normalized positions:
CTK:  ROC-AUC 0.608
ELK:  ROC-AUC 0.179
RTK:  ROC-AUC 0.305
```

What it means. ELK (0.18) and RTK (0.31) are far *below* 0.5. That is not a model that learned errors backwards. It is a side effect of pooling predictions from different folds when errors are clustered in a few people. When an error-heavy person is held out, the training set has fewer errors, so that model gives everyone lower error probabilities, including that person's error seconds. The opposite happens for mostly-correct people. Pooling then ranks error seconds low. **Lesson: with person-clustered labels, pooled scores across folds can be misleading.** This also affected the recall and precision numbers in steps 5 and 6.

Step 8 · Score inside each person

Why. The fair question is: within one person's own recordings, does the model give their error seconds a higher error probability than their correct seconds? That is also what a feedback app needs. Only people with both classes can be scored.

CODE

```
def validate_auc_per_person(table, exercise):
    ex = table[table["exercise_code"] == exercise]
    aucs = []
    for person in ex["participant_key"].unique():
        val = ex[ex["participant_key"] == person]
        if val["target"].nunique() < 2:                # needs both correct and error
            continue
        train = ex[ex["participant_key"] != person]
        model = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                       min_samples_leaf=2, random_state=0, n_jobs=-1)
        model.fit(train[feature_names], train["target"])
        prob = model.predict_proba(val[feature_names])[:, 1]
        auc = roc_auc_score(val["target"], prob)          # AUC within this one person
        aucs.append(auc)
        print(f"   {person}: {int(val['target'].sum())} error / "
              f"{int((val['target'] == 0).sum())} correct seconds -> AUC {auc:.3f}")
    print(f"{exercise}: mean within-person AUC {np.mean(aucs):.3f} ({len(aucs)} people)")
```

CODE EXPLAINED

Code	What it does
`if val["target"].nunique() < 2: continue`	If the held-out person has only one class (e.g. no errors), AUC cannot be computed. *continue* skips to the next person.
`aucs.append(auc)`	Adds one number to the end of the list.
`np.mean(aucs)`	Average of all the per-person AUCs.

OUTPUT

```
===== Raw positions =====                    ===== Normalized positions =====
CTK  1A_R1  0.173   1A_R2 0.527   1A_R3 0.649   CTK  1A_R1  0.524   1A_R2 0.596   1A_R3 0.688
     1A_R4  0.670   1A_R5 0.390   1A_R6 0.477        1A_R4  0.467   1A_R5 0.785   1A_R6 0.186
     2A_S1  0.629   2A_S2 0.952   2A_S3 0.639        2A_S1  0.365   2A_S2 0.673   2A_S3 0.647
     mean 0.567 (9 people)                          mean 0.548 (9 people)
ELK  1A_R1  0.639   1A_R3 0.744   1A_R4 0.507   ELK  1A_R1  0.504   1A_R3 0.598   1A_R4 0.489
     1A_R5  0.564   2A_S1 0.278                      1A_R5  0.529   2A_S1 0.353
     mean 0.547 (5 people)                          mean 0.495 (5 people)
RTK  1A_R1  0.882   1A_R3 0.800                 RTK  1A_R1  0.633   1A_R3 0.495
     2A_S3  0.422   2A_S6 0.586                      2A_S3  0.364   2A_S6 0.529
     mean 0.673 (4 people)                          mean 0.505 (4 people)

(Output rearranged side by side to save space; numbers unchanged.)
```

What it means. Measured inside each person, positions give scores only slightly above a coin flip. Normalized positions are at chance for all three exercises. Per-person values swing widely (0.17 to 0.95) because many people have very few error seconds (R5: 3, S2: 4). **Conclusion: a one-second summary of joint coordinates does not separate errors from correct movement.** Each error is about *how far* the body moves, which the model would have to work out from 75 separate coordinates.

Step 9 · Error-specific joint angles

Why. Calculate the angles the errors are named after, using the same formulas as `features.py` in `keraal_pipeline`. Angles do not depend on where someone stands or how tall they are.

Angle	Error it describes	What it measures
Left / right shoulder elevation	CTK: arms not raised enough	Upper arm vs trunk; 0° = arm down, 180° = straight up
Trunk side tilt	ELK: flank stretch not deep enough	Trunk lean left/right from vertical
Torso rotation	RTK: not rotating enough	Shoulder line vs hip line, seen from above

CODE

```
def angle_between(v1, v2):
    cos = (v1 * v2).sum(axis=1) / (np.linalg.norm(v1, axis=1) * np.linalg.norm(v2, axis=1))
    return np.degrees(np.arccos(np.clip(cos, -1, 1)))

def add_angles(df):
    df = df.copy()
    def get(joint):
        return df[[f"{joint}_x", f"{joint}_y", f"{joint}_z"]].values
    trunk_down = get("SpineBase") - get("SpineShoulder")
    df["shoulder_elev_left"] = angle_between(get("ElbowLeft") - get("ShoulderLeft"), trunk_down)
    df["shoulder_elev_right"] = angle_between(get("ElbowRight") - get("ShoulderRight"), trunk_down)
    trunk_up = get("SpineShoulder") - get("SpineBase")
    df["trunk_side_tilt"] = np.degrees(np.arctan2(trunk_up[:, 0], trunk_up[:, 1]))
    shoulders = get("ShoulderRight") - get("ShoulderLeft")
    hips = get("HipRight") - get("HipLeft")
    rotation = np.degrees(np.arctan2(shoulders[:, 2], shoulders[:, 0])
                          - np.arctan2(hips[:, 2], hips[:, 0]))
    df["torso_rotation"] = (rotation + 180) % 360 - 180
    return df

angle_columns = ["shoulder_elev_left", "shoulder_elev_right", "trunk_side_tilt", "torso_rotation"]

def make_second_features_cols(df, cols):          # same as step 4, for any columns
    grouped = df.groupby(["sample_id", "second_idx"])
    stats = grouped[cols].agg(["mean", "std", "min", "max"])
    stats.columns = [f"{col}_{stat}" for col, stat in stats.columns]
    info = grouped[["participant_key", "exercise_code", "target"]].first()
    return info.join(stats).reset_index()

train_sec_angle = make_second_features_cols(add_angles(train_data), angle_columns)
feature_names = [c for c in train_sec_angle.columns if c not in info_columns]
for exercise in ["CTK", "ELK", "RTK"]:
    validate_auc_per_person(train_sec_angle, exercise)
```

CODE EXPLAINED

Code	What it does
`angle_between(v1, v2)`	Angle between two directions using cos θ = (v1 · v2) / (|v1| × |v2|). *np.clip* keeps the value between −1 and 1 (tiny rounding errors could break arccos), *np.arccos* gives the angle in radians and *np.degrees* converts to degrees.
`get("ElbowLeft") - get("ShoulderLeft")`	Subtracting two joint positions gives the direction from one to the other: here the upper arm. trunk_down = pelvis − top of spine points down the back.
`shoulder_elev = angle(upper arm, trunk_down)`	0° = arm hanging down along the body, 90° = arm horizontal, 180° = arm straight up.
`np.arctan2(trunk_up[:, 0], trunk_up[:, 1])`	Angle of the trunk away from vertical towards the side (x = left/right, y = up). 0° = upright; +/− = leaning right/left.
`arctan2(shoulders z, x) − arctan2(hips z, x)`	Seen from above (x–z plane): direction of the shoulder line minus direction of the hip line. 0° = shoulders and hips face the same way; larger = more torso twist.
`(rotation + 180) % 360 - 180`	Keeps angles between −180° and 180°. % is the remainder after division. Example: 350° → (530 % 360) − 180 = −10°.
`make_second_features_cols(df, cols)`	Same as step 4, but we can choose which columns to summarise (here the 4 angles → 4 × 4 = 16 features).

OUTPUT

```
Angle training table: (4302, 21)
                     shoulder_elev_left_max  shoulder_elev_right_max  trunk_side_tilt_max  torso_rotation_max
CTK          0                         84.7                     83.1                  1.7                 1.8
             1                         91.2                     87.8                  0.4                 1.2
ELK          0                         38.3                     91.2                 -0.6                 2.0
             1                         39.2                    121.8                 -0.7                 0.1
RTK          0                         80.4                     81.1                  1.1                 3.3
             1                         77.4                     75.9                  0.7                -0.6

Mean within-person AUC:  CTK 0.660 (9 people) | ELK 0.540 (5) | RTK 0.596 (4)
```

What it means. CTK improves (0.55 → 0.66). Two warning signs remain. (1) Tilt and rotation are only about 0–3°, far smaller than a real flank stretch or torso rotation. (2) In CTK, error seconds show *higher* arm elevation than correct seconds, the opposite of “arms not raised enough”. A likely reason for (2) is timing: experts mark errors only while the person is moving, whereas “correct” seconds also include rest with arms down.

Step 10 · Align the heading and check the angles against keraal_pipeline

Why. `preprocess.py` in `keraal_pipeline` rotates each recording so the hips face the camera. Without it, a side bend by someone standing slightly turned shows up in the depth direction and is missed. This step is a check of our own code: if whole-recording angles match the numbers in `keraal_pipeline`, the code is correct.

CODE

```
def align_heading(df):
    df = df.copy()
    hip_x = df["HipRight_x"] - df["HipLeft_x"]
    hip_z = df["HipRight_z"] - df["HipLeft_z"]
    hip_angle = np.arctan2(hip_z, hip_x)                              # hip line seen from above
    mean_sin = np.sin(hip_angle).groupby(df["sample_id"]).transform("mean")
    mean_cos = np.cos(hip_angle).groupby(df["sample_id"]).transform("mean")
    theta = np.arctan2(mean_sin, mean_cos)                            # one angle per recording
    c, s = np.cos(theta), np.sin(theta)
    for joint in joints:                                              # rotate around vertical axis
        x = df[f"{joint}_x"].copy()
        z = df[f"{joint}_z"].copy()
        df[f"{joint}_x"] = x * c + z * s
        df[f"{joint}_z"] = -x * s + z * c
    return df

check = data[data["group"].isin(["1A", "2A"]) & data["evaluation"].isin(["Correct", "Incorrect"])]
check = add_angles(align_heading(check))
check["shoulder_elev"] = (check["shoulder_elev_left"] + check["shoulder_elev_right"]) / 2
rec = check.groupby(["sample_id", "exercise_code", "evaluation"]).agg(
    elev_max=("shoulder_elev", "max"),
    tilt_max=("trunk_side_tilt", "max"), tilt_min=("trunk_side_tilt", "min"),
    rot_max=("torso_rotation", "max"), rot_min=("torso_rotation", "min")).reset_index()
rec["tilt_range"] = rec["tilt_max"] - rec["tilt_min"]
rec["rot_range"] = rec["rot_max"] - rec["rot_min"]
print(rec.groupby(["exercise_code", "evaluation"])[["elev_max", "tilt_range", "rot_range"]].median().round(1))
```

CODE EXPLAINED

Code	What it does
`hip_angle = np.arctan2(hip_z, hip_x)`	Direction of the line from left hip to right hip, seen from above, in every frame.
`np.sin(...).groupby(...).transform("mean") / np.arctan2(mean_sin, mean_cos)`	A *circular mean* of angles per recording. A plain average fails near ±180°: the average of 179° and −179° should be 180°, not 0°. Averaging sin and cos and converting back gets it right.
`x * c + z * s, -x * s + z * c`	The formula for rotating a point around the vertical (y) axis by angle theta (c = cos theta, s = sin theta). After this, every recording's hips face the camera.
`x = df[...].copy()`	Saves the old x before it is overwritten, because the new z still needs the old x.
`.agg(elev_max=("shoulder_elev", "max"), ...)`	*Named aggregation*: creates a column called elev_max that holds the max of shoulder_elev for each recording, and so on.
`rec["tilt_max"] - rec["tilt_min"]`	Range = highest minus lowest: how far the trunk travelled from one side to the other in the recording.
`...median().round(1)`	Median per exercise and rating, rounded to 1 decimal.

OUTPUT

```
                          elev_max  tilt_range  rot_range
exercise_code evaluation
CTK           Correct        119.6         5.3       12.6
              Incorrect      101.5         4.6        8.3
ELK           Correct         92.4        24.1       16.9
              Incorrect       91.8        13.7        9.7
RTK           Correct         98.2         6.7       47.5
              Incorrect       84.0         7.2       15.6
```

Exercise	Angle	Correct	Incorrect	keraal_pipeline
CTK	peak arm elevation	**119.6°**	101.5°	121.6° vs 101.7° ✓
ELK	side tilt range	**24.1°**	13.7°	43.5° vs 20.8° ✓ (same direction)
RTK	rotation range	**47.5°**	15.6°	48.0° vs 21.2° ✓

What it means. The angle code is correct: CTK and RTK match the pipeline's numbers almost exactly, and ELK points the same way (smaller, probably because the pipeline used only recordings both experts agreed on, and smoothed the data). Clinically, **each error changes the angle it is named after**: about 18° less arm elevation, about 10° less side tilt, about 30° less rotation. Each angle mostly differs only in its own exercise. So the information is clearly there *per recording*.

Step 11 · Per-second angle features with heading alignment

CODE

```
train_sec_angle = make_second_features_cols(add_angles(align_heading(train_data)), angle_columns)
test_sec_angle = make_second_features_cols(add_angles(align_heading(test_data)), angle_columns)
feature_names = [c for c in train_sec_angle.columns if c not in info_columns]
for exercise in ["CTK", "ELK", "RTK"]:
    validate_auc_per_person(train_sec_angle, exercise)
```

CODE EXPLAINED

Code	What it does
`add_angles(align_heading(train_data))`	Functions run from the inside out: first rotate the skeleton to face the camera, then compute the angles, then (outer call) summarise each second.

OUTPUT

```
CTK  1A_R1 0.821  1A_R2 0.611  1A_R3 0.693  1A_R4 0.645  1A_R5 0.812  1A_R6 0.197
     2A_S1 0.981  2A_S2 0.827  2A_S3 0.925                  -> mean 0.723 (9 people)
ELK  1A_R1 0.408  1A_R3 0.359  1A_R4 0.661  1A_R5 0.700  2A_S1 0.503  -> mean 0.526 (5 people)
RTK  1A_R1 0.662  1A_R3 0.300  2A_S3 0.412  2A_S6 0.511           -> mean 0.471 (4 people)

Per-second average of trunk_side_tilt_max and torso_rotation_max: still about 0-3 degrees
```

What it means. CTK improves clearly (0.72; 8 of 9 people above 0.5). R6 is the exception every time. ELK and RTK stay at chance for two reasons. (1) Tilt and rotation have a direction (left is negative, right positive), so the “max” of a second spent leaning left is still near 0. What matters is *how far*, regardless of direction. (2) Errors are about the *peak* of a repetition, and one second taken at the start of a bend looks the same whether the person later bends fully or not.

Step 12 · Absolute angles and a 5-second context window (final features)

Why. Add the absolute tilt and rotation (size without sign). For every second, also give the model a summary of the 5 seconds around it (2 before, itself, 2 after): the highest max and lowest min in that window capture the peak of the current repetition. Predictions are still per second, with a 2-second delay.

CODE

```
def add_angles_abs(df):
    df = add_angles(df)
    df["trunk_side_tilt_abs"] = df["trunk_side_tilt"].abs()
    df["torso_rotation_abs"] = df["torso_rotation"].abs()
    return df

angle_columns_abs = angle_columns + ["trunk_side_tilt_abs", "torso_rotation_abs"]

def add_context(sec_table, window=5):
    t = sec_table.sort_values(["sample_id", "second_idx"]).copy()
    cols = [c for c in t.columns if c not in info_columns]
    g = t.groupby("sample_id")                                  # never mix recordings
    for c in cols:
        if c.endswith("_max"):
            t[c + "_ctx"] = g[c].transform(lambda s: s.rolling(window, center=True, min_periods=1).max())
        elif c.endswith("_min"):
            t[c + "_ctx"] = g[c].transform(lambda s: s.rolling(window, center=True, min_periods=1).min())
        else:
            t[c + "_ctx"] = g[c].transform(lambda s: s.rolling(window, center=True, min_periods=1).mean())
    return t

train_sec_ctx = add_context(make_second_features_cols(add_angles_abs(align_heading(train_data)), angle_columns_abs))
test_sec_ctx = add_context(make_second_features_cols(add_angles_abs(align_heading(test_data)), angle_columns_abs))
feature_names = [c for c in train_sec_ctx.columns if c not in info_columns]
for exercise in ["CTK", "ELK", "RTK"]:
    validate_auc_per_person(train_sec_ctx, exercise)
```

CODE EXPLAINED

Code	What it does
`.abs()`	Absolute value: drops the sign. −25° and +25° both become 25°, so a deep bend to either side counts the same.
`sort_values(["sample_id", "second_idx"])`	Puts seconds in time order inside each recording, which the window needs.
`.rolling(5, center=True, min_periods=1).max()`	A moving window of 5 rows centred on each second (2 before, itself, 2 after) and its maximum. At the start or end of a recording the window just uses the rows that exist (min_periods=1).
`g[c].transform(lambda s: ...)`	Runs the window separately for each recording, so seconds from different recordings are never mixed. *lambda s: ...* is a tiny one-line function applied to each recording's column s.
`t[c + "_ctx"]`	New column name with _ctx added, e.g. shoulder_elev_left_max_ctx = highest arm elevation in the 5-second window.

OUTPUT

```
Context training table: (4302, 53)

CTK  1A_R1 0.825  1A_R2 0.503  1A_R3 0.787  1A_R4 0.645  1A_R5 0.938  1A_R6 0.152
     2A_S1 0.985  2A_S2 0.901  2A_S3 0.850                  -> mean 0.732 (9 people)
ELK  1A_R1 0.675  1A_R3 0.428  1A_R4 0.682  1A_R5 0.620  2A_S1 0.503  -> mean 0.582 (5 people)
RTK  1A_R1 0.681  1A_R3 0.255  2A_S3 0.862  2A_S6 0.589           -> mean 0.597 (4 people)
```

What it means. ELK (0.53 → 0.58) and RTK (0.47 → 0.60) improve, and CTK stays strong (0.73). The model now finds error information inside a person, most clearly for CTK. ELK and RTK signals are weak and rest on 5 and 4 people. R6 is still ranked wrongly in CTK (0.15). **This version is frozen as the final model.** We tried five feature versions on 1A/2A only, so Group 3 was never used for any choice.

Step 13 · Train on all of 1A + 2A, test once on Group 3

Why. This is the real test: new people, a different recording setup. Group 3 labels cover whole recordings, so we average the per-second error probabilities within each recording and score with recording-level ROC-AUC. AUC is used instead of F1 because the 50% cut-off was learned on data with 5–16% errors, while Group 3 has 50–67% error recordings.

CODE

```
def test_on_g3(train_table, test_table, exercise):
    tr = train_table[train_table["exercise_code"] == exercise]
    te = test_table[test_table["exercise_code"] == exercise].copy()
    model = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                   min_samples_leaf=2, random_state=0, n_jobs=-1)
    model.fit(tr[feature_names], tr["target"])
    te["prob"] = model.predict_proba(te[feature_names])[:, 1]        # P(error) per second
    rec = te.groupby("sample_id").agg(target=("target", "first"),
                                      prob=("prob", "mean")).reset_index()   # per recording
    auc = roc_auc_score(rec["target"], rec["prob"])
    print(f"{exercise}: {len(rec)} recordings | "
          f"mean prob correct {rec[rec['target'] == 0]['prob'].mean():.3f} | "
          f"mean prob error {rec[rec['target'] == 1]['prob'].mean():.3f} | ROC-AUC {auc:.3f}")
    return rec

g3_results = {}
for exercise in ["CTK", "ELK", "RTK"]:
    g3_results[exercise] = test_on_g3(train_sec_ctx, test_sec_ctx, exercise)
```

CODE EXPLAINED

Code	What it does
`te["prob"] = model.predict_proba(...)[:, 1]`	Stores the error probability of every Group 3 second in a new column.
`groupby("sample_id").agg(target=("target", "first"), prob=("prob", "mean"))`	One row per recording: its label (same for all its seconds) and the average error probability of its seconds.
`g3_results[exercise] = ...`	Saves each exercise's per-recording results in a dictionary so step 14 can reuse them without retraining.

OUTPUT

```
CTK: 135 recordings | mean prob correct 0.191 | mean prob error 0.263 | ROC-AUC 0.713
ELK: 90 recordings | mean prob correct 0.221 | mean prob error 0.314 | ROC-AUC 0.768
RTK: 135 recordings | mean prob correct 0.030 | mean prob error 0.105 | ROC-AUC 0.768
```

What it means. For every exercise, error recordings get a higher average error probability than correct ones. An error recording outranks a correct one 71–77% of the time, for three people the model never saw. The probabilities are low overall (0.03–0.31) because the model learned from data where errors are rare. That does not matter for AUC, which only uses the ranking.

Step 14 · Is it luck? Does it hold per person and per error type?

Why. With 90–135 recordings from 3 people, we must check: (1) chance level, by shuffling the labels within each person 1,000 times; (2) the AUC for each person separately; (3) which error types the model catches. No retraining is needed.

CODE

```
labels = test_data.drop_duplicates("sample_id").set_index("sample_id")[["participant_key", "error_type"]]
rng = np.random.default_rng(0)                                          # repeatable shuffles

for exercise, rec in g3_results.items():
    rec = rec.join(labels, on="sample_id")
    observed = roc_auc_score(rec["target"], rec["prob"])
    null_aucs = []
    for _ in range(1000):                                               # shuffle within each person
        shuffled = rec.groupby("participant_key")["target"].transform(lambda s: rng.permutation(s.values))
        null_aucs.append(roc_auc_score(shuffled, rec["prob"]))
    null_aucs = np.array(null_aucs)
    p_value = (1 + (null_aucs >= observed).sum()) / (1 + len(null_aucs))
    print(f"{exercise}: AUC {observed:.3f} | chance 95th percentile "
          f"{np.percentile(null_aucs, 95):.3f} | p = {p_value:.3f}")
    for person, part in rec.groupby("participant_key"):                 # per person
        print(f"   {person}: AUC {roc_auc_score(part['target'], part['prob']):.3f}")
    print("   mean prob by error type:", rec.groupby("error_type")["prob"].mean().round(3).to_dict())
```

CODE EXPLAINED

Code	What it does
`labels = test_data.drop_duplicates("sample_id").set_index("sample_id")[...]`	A small lookup table: for each recording, its participant and file-name error type. *set_index* makes sample_id the row name so *join* can match on it.
`np.random.default_rng(0)`	A random number generator with a fixed seed, so the shuffles are the same every time you run it.
`rng.permutation(s.values)`	Returns the labels in a random order. Done inside each person (groupby participant), so each person keeps the same number of correct and error recordings.
`p_value = (1 + #null ≥ observed) / (1 + 1000)`	The share of shuffles that scored at least as well as the real result. The +1 counts the real result itself, so p can never be exactly 0; the smallest possible value is 1/1001 ≈ 0.001.
`np.percentile(null_aucs, 95)`	The value that 95% of the shuffled AUCs stay below. A real AUC above it is unusual for luck.
`.to_dict()`	Turns the small result into a dictionary so it prints on one line.

OUTPUT

```
CTK: AUC 0.713 | chance 95th percentile 0.578 | p = 0.001
   3_P1: AUC 0.844
   3_P2: AUC 0.702
   3_P3: AUC 0.667
   mean prob by error type: {'Error1': 0.316, 'Error3': 0.211, 'none': 0.191}

ELK: AUC 0.768 | chance 95th percentile 0.592 | p = 0.001
   3_P1: AUC 0.933
   3_P2: AUC 0.476
   3_P3: AUC 0.920
   mean prob by error type: {'Error2': 0.314, 'none': 0.221}

RTK: AUC 0.768 | chance 95th percentile 0.561 | p = 0.001
   3_P1: AUC 0.927
   3_P2: AUC 0.909
   3_P3: AUC 0.811
   mean prob by error type: {'Error1': 0.059, 'Error2': 0.151, 'none': 0.03}
```

What it means.

- **Not luck.** p = 0.001 for every exercise (the smallest possible with 1,000 shuffles). The real AUCs are well above what chance produced (0.56–0.59).
- **Per person.** RTK is the most consistent (0.81–0.93 for all three). CTK is above chance for all three (0.67–0.84). ELK is very strong for P1 and P3 (about 0.93) but at chance for P2 (0.48), so it rests on 2 of 3 people.
- **CTK error types.** The model catches Error1 (0.316 vs 0.191 for correct) but barely Error3 (0.211). `keraal_pipeline` merges them because both are described as “arms not raised enough”, yet the model treats them very differently. Possible reasons: Error3 appears in training only for R2, R3 and R6 (and R6 is always mis-ranked), or Error3 is a different movement. The metatable notes that error-code meanings are not fully confirmed.
- **RTK error types.** Both errors are detected. Error2 (“torso rotation not sufficient”) stands out most, which fits the rotation feature.

Limitations

- **Few people.** 12 people for training/validation (only 4–9 per exercise have both classes) and 3 people for testing.

- **Acted errors.** Group 3 errors were acted, probably more obviously than real patient errors (`RESEARCH_IDEAS.md`, idea A). Group 3 may be an easier test than real patients.
- **Trial confound not yet checked.** In Group 3 each session pairs correct recordings with one error type. The pipeline found that sessions can be told apart from the skeleton. Our model was trained on 1A/2A, so it cannot have learned Group 3 session cues, but a session shift could still move the angles.
- **Timing in the labels.** Experts mark errors only while the person moves; “correct” seconds also include rest. Part of the per-second signal may be “moving vs resting”.
- **One model type and one random seed.** No logistic-regression check or repeat seeds yet.
- **Direction.** Train on real, test on acted: the reverse of the pre-registered analysis, so exploratory.

Next steps

- **Within-trial check on Group 3:** compute the AUC inside each session (correct + error from the same trial), to rule out the trial confound.
- **CTK Error3:** compare arm angles for Error1 vs Error3, and look at R6, before deciding whether to merge them.
- **ELK, participant P2:** check whether P2's flank stretch or camera position differs.
- **Robustness:** repeat with seeds 0–4 and a logistic-regression model; the conclusion should not change.
- **BlazePose:** run the same pipeline on `model_input_blazepose.csv` and compare it with Kinect.